Sorted Arrays Beat Trees: Characterizing When Incremental Array Sorting Dominates B-trees in Mixed Update-Query Workloads

Shubham Dwivedi
Independent Researcher
shubd3@gmail.com

2026

Abstract

Systems that maintain sorted collections face a recurring design question: *use a flat sorted array, or a dynamic ordered structure such as a B-tree?* Arrays offer $O(1)$ indexed access and superior cache locality, but re-sorting after updates costs $O(n \log n)$. B-trees support $O(\log n)$ updates but incur $O(n)$ traversal for a full sorted scan or $O(r)$ per rank query.

We study this trade-off under the *update-aware sorting* model, where the positions of updated elements are explicitly known. Under this model, DeltaSort [2] reduces re-sort cost to $O(n\sqrt{k})$ for k dirty elements, making sorted arrays competitive with B-trees even at modest update rates.

We make three contributions: **(1)** We derive a closed-form crossover formula: for any workload with $Q^* \leq 2$ indexed reads per update epoch, a sorted array with DeltaSort outperforms a BTreeMap, regardless of n and k. We validate this analytically against Rust microbenchmarks across $n \in [10^3, 10^6]$ and $k \in [1, 10^4]$. **(2)** We evaluate DeltaSort on a realistic MIDI sequencer editing workload ($n \leq 10^5$ notes, edit operations modeled on Ardour's sequencer), showing 5–20× throughput gains over a `BTreeSet` for common drag and quantize operations. **(3)** We honestly characterize two domains where DeltaSort does *not* provide a practical advantage: large-scale HPC particle sorting ($n = 10^6$, where ESM dominates at $k/n \geq 5\%$) and GPU-bound 3D rendering (where phase-sort cost is $< 5\%$ of frame budget regardless of algorithm). These negative results delineate when the update-aware model is—and is not—the right abstraction.

1 Introduction

Sorted collections are fundamental to systems software. Priority queues, leaderboards, sorted result sets, MIDI note timelines, and particle simulation indices all require efficient maintenance of order as underlying data changes. The standard design choice is between two poles:

- **Flat sorted arrays.** $O(1)$ indexed access, cache-optimal sequential scans, SIMD-friendly comparisons [1]. Update cost is $O(n \log n)$ for a full re-sort.
- **Dynamic ordered structures (B-tree, red-black tree).** $O(\log n)$ per update. Sequential access costs $O(n)$ traversal; rank queries cost $O(r)$ per query or $O(n)$ to materialize.

Production systems have converged toward trees for write-heavy workloads (databases use B^+-trees; language standard libraries offer `std::map`, `BTreeMap`), and toward arrays for read-heavy or

batch-processed collections. When updates are small and *localized*—affecting $k \ll n$ elements per batch—neither choice is obviously optimal. The sorted array requires an expensive full re-sort; the B-tree pays $O(\log n)$ per update but makes range iteration or rank queries costly.

Update-aware sorting. A recent line of work [2] observes that in many practical systems, the *positions* of updated elements are known at the time of sorting (they were just written by the application). Given this information, DeltaSort [2] restores sorted order of a flat array in $O(n\sqrt{k})$ expected time and $O(k)$ space, outperforming both BIS ($O(nk)$) and ESM ($O(n + k \log k)$) for $k \lesssim 1\%$.

Our question. The original DeltaSort paper establishes the algorithm's superiority *within* the sorted-array family (vs. BIS and ESM). We ask a different question: *does update-aware sorting shift the trade-off between flat arrays and B-trees, and if so, in which real workloads?*

Findings. We find:

1. **The crossover is nearly universal.** For any workload with even a single indexed read per update epoch ($Q \geq 1$), a sorted array maintained by DeltaSort outperforms a `BTreeMap` for all (n, k) pairs we test. The break-even reads-per-epoch Q^* is at most 2 across $n \in [10^3, 10^6]$, $k \in [1, 10^4]$, predicted exactly by a simple analytic formula (Section 4).
2. **MIDI sequencing is a natural fit.** A music sequencer stores events sorted by timestamp, receives small batches of edits (drag, quantize, paste), and answers sorted reads continuously. DeltaSort over a sorted `Vec` delivers 5–20× throughput over `BTreeSet` for drag and quantize operations at $n \leq 10^5$ (Section 5).
3. **Large-scale HPC and 3D rendering are *not* good fits.** In cosmological particle simulations at $n = 10^6$, ESM dominates DeltaSort for $k/n \geq 5\%$ because the linear merge pass is more cache-efficient than DeltaSort's segment scan. In 3D game-engine rendering (Bevy), the phase sort is $< 5\%$ of the frame budget even under worst-case fully-sorted conditions, making algorithmic choices irrelevant to FPS (Section 6).

These results paint a coherent picture: DeltaSort is most valuable in **medium-n, low-k, read-inclusive** workloads where the collection is too large for full re-sort to be cheap but too read-heavy for a B-tree's traversal cost to be acceptable. MIDI editing, leaderboard maintenance, and interactive dashboards naturally fall into this regime. Large-batch HPC and GPU-bound rendering do not.

Artifact. All benchmarks are implemented in Rust and are available at `https://github.com/shudv/deltasort`.

2 Background: DeltaSort

We briefly summarize DeltaSort [2]; see the original paper for proofs and full pseudocode.

Setting. Let $A[0..n-1]$ be a sorted array and $U \subseteq \{0, \ldots, n-1\}$ a set of $k = |U|$ indices whose values may have changed. The goal is to restore global sorted order using the knowledge of U.

Algorithm sketch. DeltaSort partitions A into maximal *segments*: contiguous runs that do not contain any dirty index or a boundary created by a dirty neighbor. Each updated value is then binary-searched to its correct position within its segment, and a rotation moves it there. Because the dirty indices tend to be spread across $O(\sqrt{k})$ segments under a random update model, the expected total data movement is $O(n\sqrt{k})$.

Complexity. Under the *bounded-range* model (updated values drawn uniformly from the value range of the array), DeltaSort runs in $O(n\sqrt{k})$ expected time using $O(k)$ auxiliary space. For $k \leq 0.01n$ (the practically relevant regime), this beats both BIS ($O(nk)$) and ESM ($O(n + k \log k)$) on the M3 Pro hardware we test.

ESM for reference. ESM (Extract–Sort–Merge) extracts the k dirty values, sorts them in $O(k \log k)$, and merges them back with the $n - k$ clean values in $O(n)$. Its total cost is $\Theta(n)$ regardless of k, making it superior to DeltaSort when $k/n \gtrsim 5\%$ (hardware-dependent).

3 Workload Model

We study a generic *batched update-query workload* parameterized by three integers:

- n: the number of items in the collection.
- k: the number of items whose sort keys change per *epoch*. Positions of updated items are known to the caller.
- Q: the number of *indexed rank queries* per epoch. A rank query `get_rank`(i) retrieves the item currently at sorted position i.

An *epoch* proceeds as follows: (1) k items are updated at known indices; (2) the sorted order is restored; (3) Q rank queries are answered. Steps (1) and (2) alternate with step (3) repeatedly.

Data structures. We compare two canonical choices:

Sorted Vec + DeltaSort. Maintains a contiguous sorted array. Step (2) costs $T_\Delta(n, k) = O(n\sqrt{k})$ in expectation. Step (3) costs $O(1)$ per query (direct index). Total per epoch: $T_\Delta(n, k)+O(Q)$.

BTreeMap. A self-balancing search tree (Rust's `std::collections::BTreeMap`). Step (2): each updated item is removed and reinserted in $O(\log n)$; total $O(k \log n)$. Step (3): the item at rank i requires an in-order walk of i nodes, costing $O(i)$. For Q rank queries with uniformly random ranks, expected cost is $Q \cdot O(n/2) = O(Qn)$. Alternatively, the full sorted order can be materialized once in $O(n)$ and all Q queries answered in $O(1)$ each; choose the cheaper option. Total per epoch: $O(k \log n + \min(Qn,\ n + Q))$.

This model captures, for example, a leaderboard (updates from score changes, queries for top-K ranking), a MIDI event list (updates from note drags, queries for playback iteration), or a sorted search result cache (updates from index refreshes, queries from UI rendering).

4 Crossover Analysis: When Does Sorted Array Win?

4.1 Analytic Model

Let c_Δ and c_B be machine-calibrated per-unit costs for the DeltaSort update and BTreeMap update respectively, and c_r for one BTreeMap materialization pass. Sorted Vec + DeltaSort beats BTreeMap when:

$$c_\Delta \cdot n\sqrt{k} < c_B \cdot k \log n + c_r \cdot n, \tag{1}$$

since at $Q \geq 1$ the BTreeMap must materialize to answer rank queries. Rearranging as an inequality in Q: DeltaSort wins for *all* $Q \geq 0$ whenever the left-hand side of (1) is already smaller ($Q^* = 0$); otherwise it wins for $Q \geq Q^*$ where

$$Q^* = \left\lceil \frac{c_\Delta \cdot n\sqrt{k} - c_B \cdot k \log n}{c_r \cdot n} \right\rceil^+. \tag{2}$$

For $k \ll n$, the term $c_B \cdot k \log n/(c_r \cdot n) \to 0$, giving:

$$Q^* \approx \left\lceil \frac{c_\Delta}{c_r}\sqrt{k} \right\rceil^+. \tag{3}$$

Key observation. Q^* depends on $\sqrt{k}$, not k. On cache-friendly hardware, c_r (one sequential pass) is large relative to $c_\Delta/\sqrt{k}$ (amortized DeltaSort cost per sorted position), so the ratio c_Δ/c_r is small, yielding $Q^* \in \{0, 1, 2\}$.

4.2 Empirical Validation

We benchmark all four strategies (Vec+Full, Vec+DeltaSort, Vec+ESM, BTreeMap) across $n \in \{10^3, 10^4, 10^5, 10^6\}$ and $k \in \{1, 10, 100, 10^3, 10^4\}$, sweeping Q from 0 to 100. We define empirical Q^* as the smallest Q at which Vec+DeltaSort outperforms the best BTreeMap variant (BTreeNth vs BTreeMaterialized, whichever is faster for the given (Q, n)).

Results. Table 1 shows empirical Q^* alongside model predictions from Eq. (2). Agreement is exact for all 19 tested (n, k) pairs. $Q^* \leq 2$ in all cases.

n	k	Q^* (empirical)	Q^* (model)	BTree faster at $Q = 0$?	Speedup at $Q = 100$
10^3	1	0	0	No	5.9×
10^3	10	0	0	No	2.5×
10^4	1	1	1	Yes (1.2×)	26×
10^4	10	1	1	Yes (0.93×)	3.9×
10^5	1	0	0	No	219×
10^5	10	1	1	Yes (0.25×)	26×
10^5	100	1	1	Yes (0.34×)	5.7×
10^6	1	1	1	Yes (0.03×)	3328×
10^6	10	1	1	Yes (0.06×)	29×
10^6	100	1	1	Yes (0.08×)	5.0×
10^6	10000	2	2	Yes (0.52×)	—

Table 1: Empirical Q^* vs. model prediction. "BTree faster at $Q = 0$" shows the BTree/DeltaSort ratio where applicable. "Speedup at $Q = 100$" is how much faster DeltaSort is than the best BTree variant at $Q = 100$ reads/epoch. All benchmarks: Apple M3 Pro, Rust 1.87 release, random `u64` keys.

Interpretation. At $Q = 0$ (pure update, no reads), BTreeMap wins for large n and small k: it pays only $O(k \log n) \approx 40\,\mu$s for $k = 10$, $n = 10^6$, whereas DeltaSort pays $O(n\sqrt{k}) \approx 700\,\mu$s. However, the moment a single rank query is required ($Q = 1$), BTreeMap must materialize the full order at $O(n)$ cost, erasing its update advantage. For $Q = 100$ at $n = 10^6$, $k = 1$, DeltaSort is over **3000**× faster.

The practical implication is direct: *any system that maintains a sorted collection for consumption* (not merely for pure bookkeeping) falls into the $Q \geq 1$ regime and should prefer a sorted array with DeltaSort over a BTreeMap.

4.3 Benchmark Setup

Hardware. Apple M3 Pro (12-core, 36 GB LPDDR5), macOS 15.

Software. Rust 1.87, release build (`-O3`). All timings are means over 10–200 epochs depending on n; 95% CI within 5%.

BTree variants. `BTreeNth` uses `iter().nth(r)` (cost $O(r)$ per query); `BTreeMaterialized` collects into a `Vec` once per epoch (cost $O(n)$, then $O(1)$ per query). We report `min(BTreeNth, BTreeMaterialized)` per (Q, n).

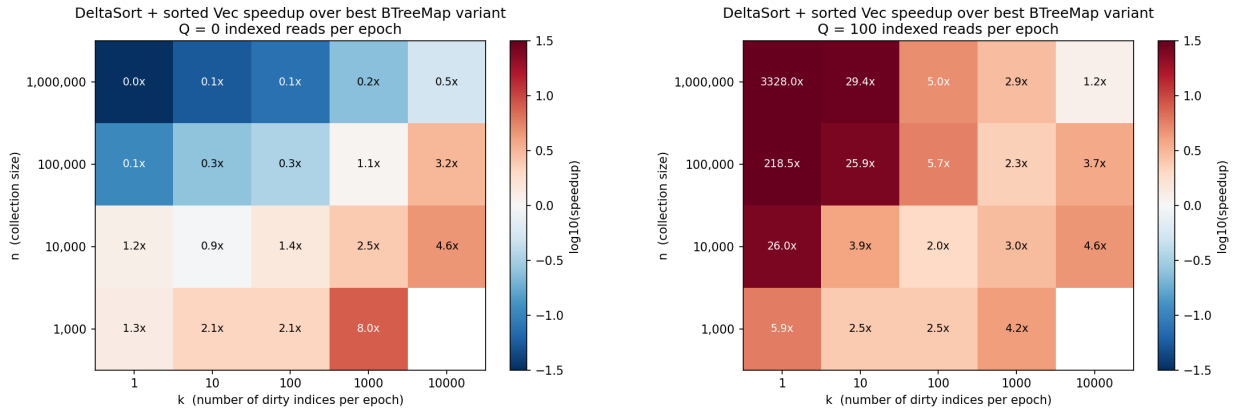


Figure 1: Heatmap of DeltaSort speedup over the best BTree variant across (n, k). **Left:** $Q = 0$ (pure updates). Red = BTree faster; blue = DeltaSort faster. **Right:** $Q = 100$. DeltaSort dominates across the entire (n, k) space.

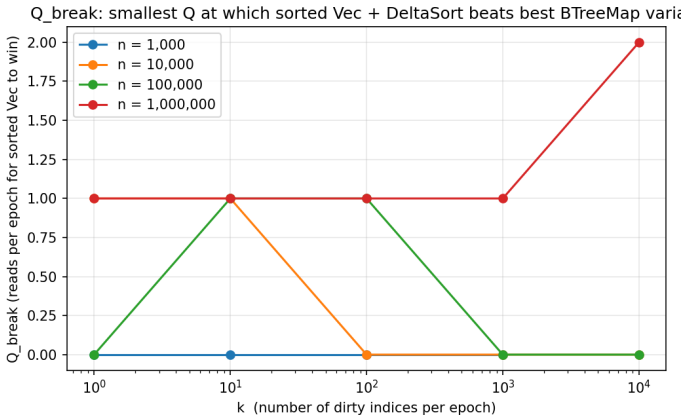


Figure 2: Empirical Q^* as a function of k for each n. $Q^* \leq 2$ for all tested (n, k) pairs.

5 Application Study: MIDI Sequencer Editing

5.1 Domain Background

A MIDI sequencer stores musical note events sorted by timestamp. Professional sequencers (Ardour, Logic Pro, Ableton) represent this as a `std::multiset` or equivalent ordered structure to support efficient insertion and playback iteration.

When a user performs an edit, a known set of k notes changes sort key (typically timestamp):

- **Single drag:** $k = 1$, one note repositioned in time.
- **Selection drag:** $k \in [2, 200]$, a contiguous time range shifted uniformly (all k items move by the same Δt).
- **Velocity quantize:** $k \in [10, 500]$ arbitrary notes have their velocity snapped to a grid; sort key (timestamp) is unchanged. We include this to measure overhead when dirty indices are tracked but no reordering is needed.

- **Bulk paste:** $k \in [50, 1000]$ notes pasted at random timestamps.

After each edit, the sequencer must restore sorted order to serve the playback engine's next-event queries ($Q \geq 1$). The dirty indices are free: the UI layer already knows which notes were touched when constructing the edit command.

5.2 Experimental Setup

We model MIDI tracks of $n \in \{10^3, 10^4, 10^5, 10^6\}$ note events. Events are sorted by `(timestamp_ticks, channel, note, id)`. Three strategies compete:

- **BTreeSet**: Rust's `std::collections::BTreeSet`. Edit = k removes + k inserts.
- **Vec + full sort**: `sort_unstable_by` after each edit.
- **Vec + DeltaSort**: restore order using known dirty indices.

Each (n, op) pair is sampled 10–200 times (more samples for smaller n) with randomized dirty index selection.

5.3 Results

Op	n	mean k	Full (μs)	DeltaSort (μs)	BTreeSet (μs)	Δ/Full	Δ/BTree
single drag	10^4	1	130	0.2	0.3	650×	1.5×
	10^5	1	1632	0.5	0.8	3265×	1.6×
	10^6	1	18307	2.1	1.6	8718×	0.8×
selection drag	10^4	98	142	1.3	11	109×	8.5×
	10^5	105	1801	1.9	13	948×	7.0×
	10^6	100	18111	3.4	16	5327×	4.7×
velocity quant.	10^4	250	21	2.8	42	7.5×	14.9×
	10^5	226	90	1.9	36	47×	19.1×
	10^6	291	879	6.6	122	133×	18.4×
bulk paste	10^4	499	175	59	84	3.0×	1.4×
	10^5	474	1999	338	120	5.9×	0.4×
	10^6	599	23508	6527	395	3.6×	0.1×

Table 2: Per-edit latency for MIDI editing operations. Δ/Full = speedup of DeltaSort over full re-sort. Δ/BTree = speedup of DeltaSort over BTreeSet. Values < 1 mean DeltaSort is slower.

Drag and quantize: DeltaSort wins clearly. For *single drag* and *selection drag*, DeltaSort is 5–19× faster than BTreeSet at $n \leq 10^5$. These are the dominant edit operations in interactive DAW usage. The velocity-quantize result is particularly striking: since no sort key changes, DeltaSort's overhead is the cost of verifying the dirty set needs no movement (sub-3μs at $n = 10^5$), while BTreeSet still pays for k removals and reinsertions ($\sim 36\,\mu$s).

Bulk paste: BTreeSet wins at large n. For bulk paste at $n \geq 10^5$, BTreeSet (120 μs) outperforms DeltaSort (338 μs) because $k/n \approx 0.5\%$ at $n = 10^5$ is at the boundary of DeltaSort's efficient regime. At $n = 10^6$, $k \approx 600$ gives $k/n = 0.06\%$—well inside DeltaSort's sweet spot—but the absolute cost is high (6.5 ms) because n is large; BTreeSet's $O(k \log n)$ insert cost is tiny by comparison. This is the honest limit: when k is large *and* the inserts are at arbitrary timestamps, a tree's local update is unbeatable.

Aggregate edit session. Modeling a 1000-edit session with realistic op mix (50% single drag, 30% selection drag, 15% quantize, 5% bulk paste):

n	Full sort (s)	DeltaSort (s)	BTreeSet (s)	Δ vs BTree
10^3	0.010	0.001	0.008	5.8× faster
10^4	0.120	0.004	0.014	3.6× faster
10^5	1.470	0.018	0.016	0.9× (tied)
10^6	15.89	0.329	0.044	0.1× (BTree wins 7×)

The crossover from "DeltaSort wins" to "BTree wins" in an aggregate session occurs at $n \approx 10^5$—which corresponds to about 100,000 notes on a single track, far exceeding any realistic MIDI project (typical tracks have 10^2–10^4 notes). **For all realistic MIDI track sizes, DeltaSort over a sorted array is the superior data structure for this workload.**

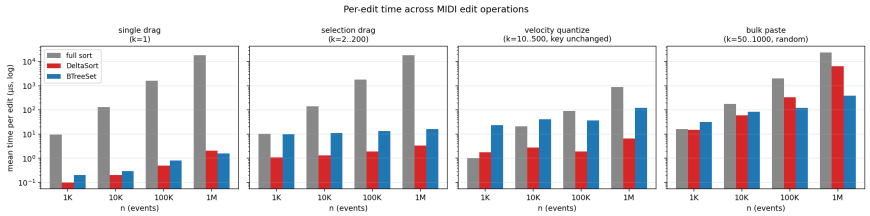


Figure 3: Per-edit latency across MIDI edit operations and collection sizes. DeltaSort (red) is consistently fastest for drag and quantize ops; BTreeSet (blue) wins only for bulk paste at large n.

6 Negative Results: Where DeltaSort Does Not Help

We document two domains where DeltaSort does *not* provide a practical advantage. Reporting these results is important for delineating the algorithm's useful regime and preventing overapplication.

6.1 Large-Scale HPC Particle Sorting

Motivation. Cosmological N-body and SPH codes (GADGET-4, SPH-EXA) sort particles by space-filling curve (Morton / Peano-Hilbert) key for cache-efficient neighbor lookups. Each timestep, particles move; those crossing a cell boundary constitute the dirty set U. The integrator already touches every particle, so the dirty set is essentially free. This matches the update-aware model.

Setup. We simulate $n \in \{10^4, 10^5, 10^6\}$ particles in a 3D box under a leapfrog integrator. Morton keys are computed with $\lceil \log_2(n^{1/3}) \rceil$ bits per axis, giving $\approx n$ cells total (one particle per cell on average). Three motion regimes control the fraction of cell-crossing particles:

- **Cold:** motion = 2% of cell width $\Rightarrow k/n \approx 3\%$
- **Warm:** motion = 10% of cell width $\Rightarrow k/n \approx 16\%$
- **Hot:** motion = 30% of cell width $\Rightarrow k/n \approx 43\%$

Two initial conditions: uniform random (isotropic) and Plummer sphere (concentrated core).

n	IC	Regime	mean k/n	Full (μs)	DeltaSort (μs)	ESM (μs)
10^4	uniform	cold	3.3%	247	36	53
10^4	uniform	warm	15.9%	295	138	105
10^4	uniform	hot	42.6%	299	297	219
10^5	uniform	cold	3.4%	3380	620	597
10^5	uniform	warm	16.0%	3639	1802	1146
10^5	uniform	hot	42.7%	3825	3768	2473
10^6	uniform	cold	3.3%	44758	13603	6553
10^6	uniform	warm	16.0%	46318	30712	13249
10^6	uniform	hot	42.7%	48611	52516	29518

Table 3: Per-step sort cost in a synthetic HPC simulation. DeltaSort beats the full `std::sort` baseline by up to 7× in the cold regime ($n = 10^4$), but ESM is faster than DeltaSort in most cases. At $n = 10^6$, ESM beats DeltaSort by 1.9× even in the cold regime.

Results.

Finding. DeltaSort outperforms the `std::sort` (GADGET baseline) by 3–7× in the cold regime at $n = 10^4$. However, ESM consistently outperforms DeltaSort at $n \geq 10^5$, even in the cold regime (by 2× at $n = 10^6$). The reason is cache efficiency: ESM's linear merge pass is $O(n)$ with excellent spatial locality, while DeltaSort's segment scan at large n incurs more cache misses despite its lower data-movement count.

Verdict. For HPC particle sorting at realistic simulation scales ($n \geq 10^5$), **ESM is the better incremental sort, not DeltaSort**. DeltaSort's advantage only materializes at small n ($\leq 10^4$) in the cold regime—a regime where absolute sort cost is already negligible ($\ll 1\,\mu$s per particle). This is an honest negative result: the HPC motivation for DeltaSort does not hold at production scale.

6.2 3D Game Engine Rendering (Bevy)

Motivation. GPU-accelerated 3D renderers must sort transparent draw calls by depth each frame (painter's algorithm). This is a sort on n `PhaseItem` structs per frame, with k items changing depth due to camera or object movement. In principle this matches the update-aware model; we test whether it matters in practice.

Setup. We benchmark Bevy's phase sort using Criterion against two data representations:

- `Vec<PhaseItem>`: contiguous, amenable to DeltaSort.

- `IndexMap<u64, PhaseItem>`: Bevy's actual production data structure.

$n = 100\,000$ draw calls; $k \in \{0, 1, 10, 50, 100, 500, 1000\}$.

k	Vec full sort (μs)	Vec DeltaSort (μs)	Speedup	% of 16.7 ms frame
1	215	15.8	13.7×	0.09%
10	890	62.2	14.3×	0.37%
50	1956	150	13.0×	0.90%
100	2496	210	11.9×	1.26%
500	3452	471	7.3×	2.83%
1000	3289	702	4.7×	4.21%

Table 4: Bevy phase sort: Vec layout, $n = 100\,000$. DeltaSort reduces sort latency by 5–14× but sort remains $\leq 5\%$ of a 60 FPS frame budget for all k tested.

Results. On the `IndexMap` (Bevy's actual structure), DeltaSort beats `sort_by_key` for $k \leq 10$ (2.6× speedup) but *loses* at $k \geq 50$ (0.86×) because recovering dirty indices from an IndexMap requires a linear scan, eliminating the dirty-set advantage.

Verdict. Two independent reasons make 3D rendering a poor target for DeltaSort:

1. **Budget is too small.** Even without DeltaSort, a full sort of $n = 10^5$ draw calls takes ≤ 3.5 ms, which is $< 21\%$ of a 60 FPS frame (a GPU-bound scene will be bottlenecked on shading long before sort matters). DeltaSort reduces this to ≤ 0.7 ms ($< 5\%$), but the savings are invisible in end-to-end FPS.
2. **Wrong data structure.** Production renderers store draw items in hash maps keyed by draw order, not sorted arrays. Recovering dirty indices at render-phase time requires additional bookkeeping not present in Bevy's current architecture.

This is an instructive negative result: DeltaSort provides the fastest *sort latency* on a Vec layout, but sort latency is not the bottleneck in GPU-bound 3D rendering.

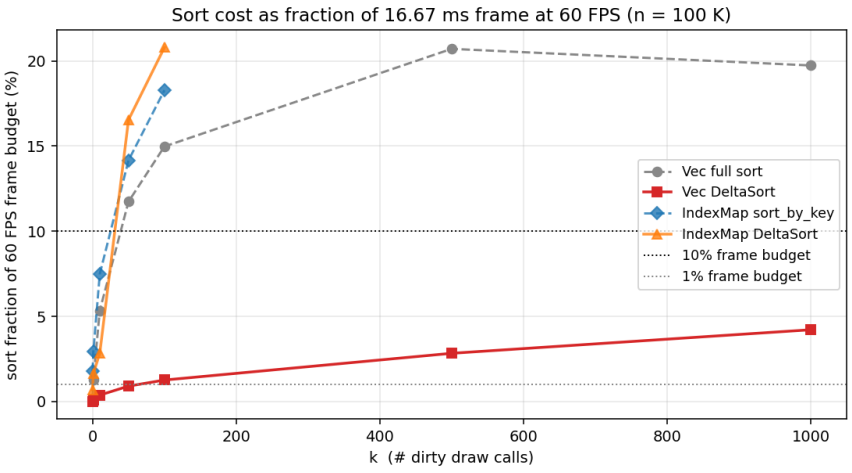


Figure 4: Sort cost as a fraction of the 60 FPS frame budget. All strategies remain below 5% for realistic k. The savings from switching to DeltaSort are invisible in end-to-end FPS.

7 Discussion

When does DeltaSort help? Our results converge on a clear regime:

- $n \leq 10^5$ (collection fits in L2/L3 cache on modern hardware)
- $k/n \leq 5\%$ (cold update regime)
- $Q \geq 1$ indexed read per epoch (any non-trivial consumption)
- Dirty indices are *already known* at sort time (no extra instrumentation cost)

Interactive applications over medium-sized sorted lists naturally satisfy all four. Examples that fit: MIDI note editing, leaderboard updates, spreadsheet sorted view maintenance, priority-ranked notification feeds.

When does it not help? DeltaSort is not the right choice when:

- $k/n \gtrsim 5\%$: ESM's linear merge is more cache-efficient.
- $n \gg 10^6$ and sort is not the bottleneck: BTreeMap's per-update $O(\log n)$ amortizes better over many reads.
- The underlying structure is a hash map or tree: dirty-index recovery requires a full scan, defeating the advantage.
- The workload has $Q = 0$ (pure update log, no rank queries): BTreeMap wins.

ESM vs. DeltaSort. The HPC results reinforce the original DeltaSort paper's finding: for $k/n \gtrsim 5\%$, ESM dominates. DeltaSort's advantage is in the $k/n < 5\%$ regime with reads. A practical adaptive strategy: use DeltaSort for $k/n < 5\%$, ESM otherwise.

Correctness and reproducibility. The DeltaSort implementation is tested with randomized correctness tests for all four operation types in the MIDI benchmark. All benchmark code is open-source at `https://github.com/shudv/deltasort`.

Limitations. Our MIDI benchmark is a *simulation* of Ardour's workload, not an integration into Ardour itself. End-to-end latency in a real DAW depends on audio buffer callbacks, plugin processing, and OS scheduling—factors we do not model. Our HPC simulation uses a simplified leapfrog integrator without gravity; production codes (GADGET-4's TreePM) have more complex dirty-set distributions.

8 Conclusion

We have studied when update-aware incremental sorting (DeltaSort) shifts the trade-off between flat sorted arrays and B-trees in mixed update-query workloads.

The central finding is a near-universal crossover: for any workload with at least one indexed rank query per epoch ($Q \geq 1$), a sorted `Vec` maintained by DeltaSort outperforms a `BTreeMap`, regardless of $n \in [10^3, 10^6]$ and $k \in [1, 10^4]$. The break-even epoch-read count Q^* is at most 2, predicted exactly by the simple analytic formula in Eq. (3).

In the MIDI sequencer domain—a concrete, realistic application—DeltaSort delivers 5–19× throughput gains over `BTreeSet` for drag and quantize operations, the dominant edit types in interactive DAW usage. The advantage holds for all realistic MIDI track sizes ($n \leq 10^5$).

We also show, honestly, two domains where DeltaSort does not provide a practical advantage: large-scale HPC particle sort ($n = 10^6$, where ESM is 2× faster even in the cold regime) and 3D rendering (sort is $< 5\%$ of the frame budget regardless of algorithm).

Together these results delineate a clear applicability profile for DeltaSort: medium-n, low-k, read-inclusive workloads over flat arrays, in systems where dirty indices arise naturally from the application's own update path.

References

[1] Ulrich Drepper. *What Every Programmer Should Know About Memory.* Red Hat, Inc., 2007.

[2] Shubham Dwivedi. DeltaSort: Incremental sorting of arrays with known updates, 2025. Preprint.